More About Classes

Prof. Ajay Bakre and Ahana Pradhan

Stack and Heap Memory

- https://circuitlabs.net/labs/c-stack-heap-memory-visualizer-learning-tool/

Memory Layout of a Program

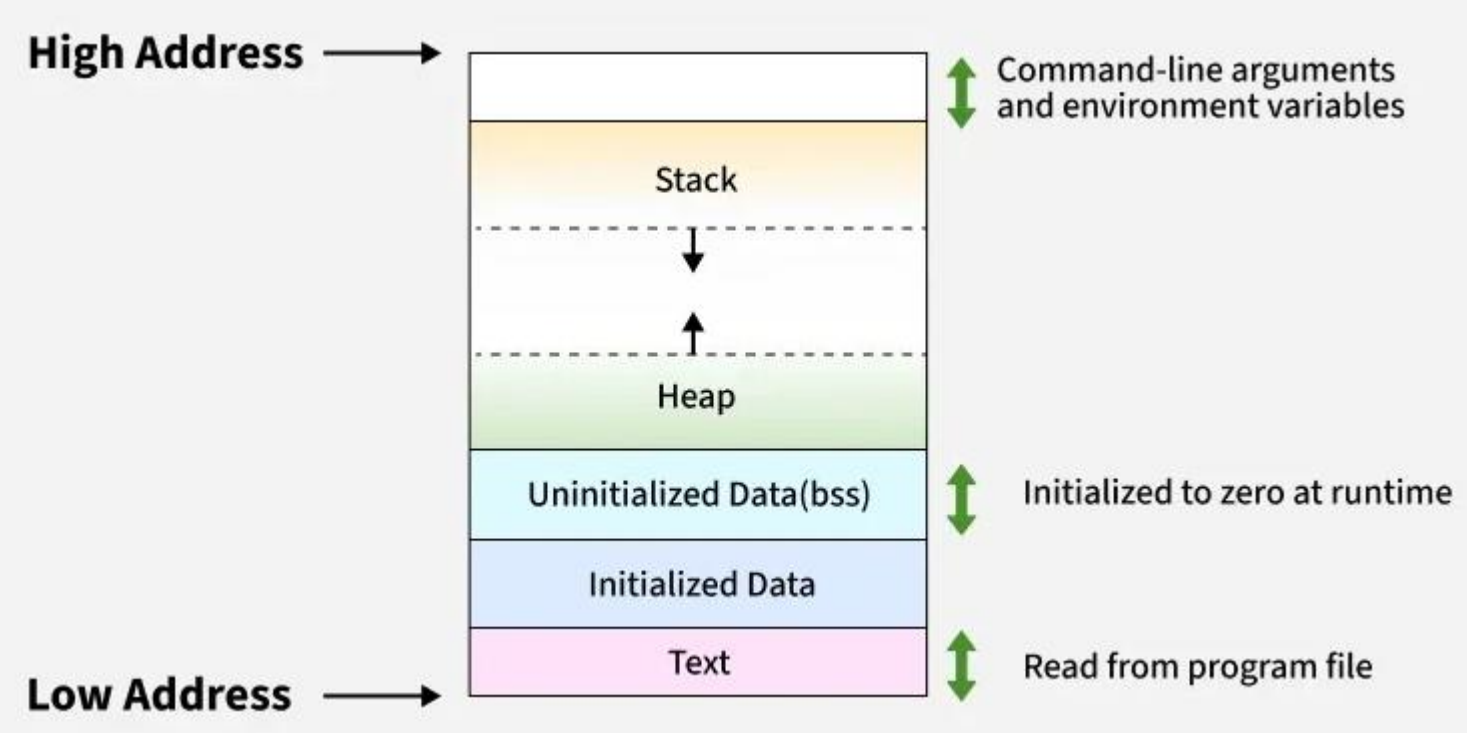


Image source: [Memory Layout of C Programs - GeeksforGeeks]

1. Local (Automatic) Storage
   - Stack
2. Static Storage
   - Data Segment
   - BSS
3. Dynamic Storage
   - Heap

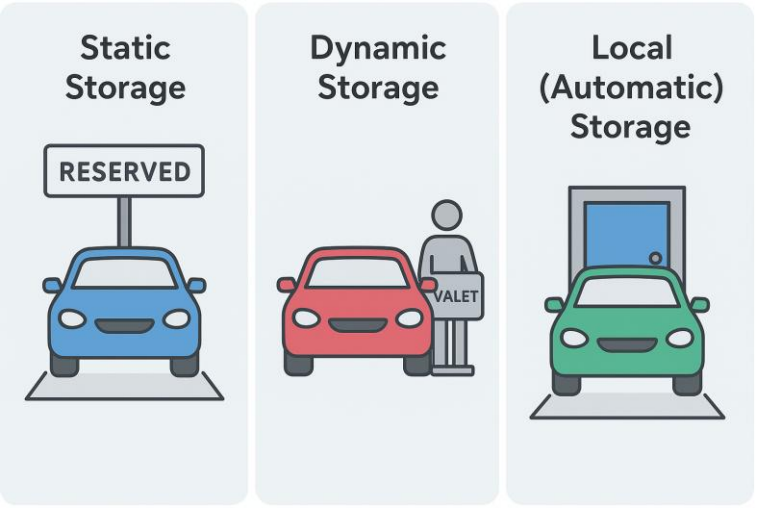


Image source: generated by ChatGPT on 14-Aug-2025

```cpp
#include<iostream>
    using namespace std;

    int global_var = 450;
    static int static_storage_var = 460;

    void showHeapMem() {
        int* heap_ptr[5];
        for (int i = 0; i<5; i++) {
            int* heap_mem = new int(40);
            heap_ptr[i] = heap_mem;
            cout << "Heap mem: " << heap_mem << endl;
        }
        for (int i = 0; i<5; i++) {
            delete heap_ptr[i];
        }
    }

    void showStackMem() {
        int* stack_ptr[5];
        for (int i = 0; i<5; i++) {
            int stack_mem = i*4;
            stack_ptr[i] = &stack_mem;
            cout << "Stack mem: " << &stack_mem << endl;
        }
    }

    int main() {
        showHeapMem();
        showStackMem();
        cout << "Static/Global Memory: " << &global_var << endl;
        cout << "Static/Global Memory: " << &static_storage_var << endl;
    }
```

```
Heap mem: 0x21f4c531a70
Heap mem: 0x21f4c531a90
Heap mem: 0x21f4c531ab0
Heap mem: 0x21f4c531ad0
Heap mem: 0x21f4c531af0
Stack mem: 0xe344bffd5c
Stack mem: 0xe344bffd5c
Stack mem: 0xe344bffd5c
Stack mem: 0xe344bffd5c
Stack mem: 0xe344bffd5c
Static/Global Memory: 0x7ff753044000
Static/Global Memory: 0x7ff753044004
```

Dynamic Memory Management

- Dynamic memory allocation is managed through heap
  - Heap is a pool of memory which is determined at runtime
  - Size and duration of memory required are also determined at run-time
- New objects are created on the heap, as required at run-time
- Freed objects are returned to the heap
- Objects on heap, pointers on stack
- All objects are freed when deleted explicitly
- Dangling pointers are invalid addresses
- Memory “leak” occurs when unused objects are not freed
  - Serious issue for long-running programs as eventually the system will run out of heap memory, e.g. a web-server “leaks” few bytes on every web request
- Garbage collection (GC) requires run-time book keeping to keep track of objects that have fallen out of scope but are still holding on to memory
  - No GC in C++ by default but some environments may support GC

Operators *new* and *delete*

- *new*
  - Allocates memory for object on heap dynamically
  - Calls constructor of the appropriate type / class
  - Returns appropriate pointer to newly created object
  - Objects in heap are nameless
- *delete*
  - Calls destructor of the appropriate type / class
  - Deallocates memory for object on heap dynamically
  - delete p - Deletes object that p points to, not p
- Death
  - Objects on heap die on using delete
  - stack variables p, q, r, s die when the scope ends

Static

static Keyword

- OOPs/static_uses.cpp at main · ahanapradhan/OOPs

Static Objects

- Following objects are initialized only ONCE and live until the end of the program
  - Global variables outside of any scope such as namespace or function or { }
  - Global variables inside a namespace scope
  - Any variables declared as “static” inside a function or class
- These have the same address throughout the life of a program and can hold state across function invocations

  e.g. void func() { static int j = 0;… } // will only be initialized on first call
- Can cause problems in a multi-threaded program
  - Synchronization / locking needed in a multi-threaded program
- Any variables not declared as “static” are local / instance variables and are initialized every single time when control reaches their initialization

  e.g. void func() { int i = 0;…}

Properties of Static Class Members

- Static storage class is different from local
- Access – has special syntax
- Static functions can access only static data
- Other (non-static) instance functions can access static & instance data

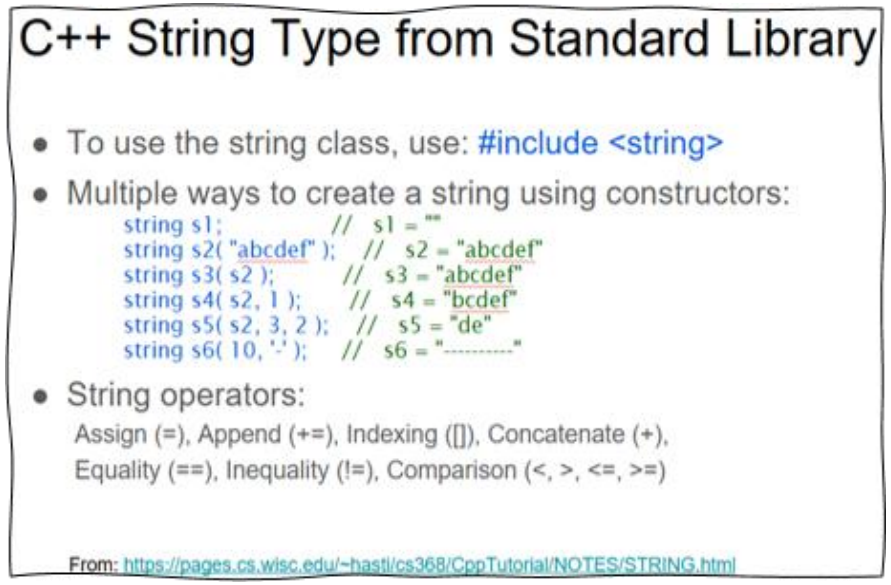
Class

Special Member Functions

- Constructors
- Destructors
- Copy Constructor
- Overloaded Assignment “=“ Operator
- Move Constructor – optional to provide efficient move operation for large objects
- Defaults for first 4 are provided by the compiler if not defined explicitly
- Destructor is needed if memory is allocated as part of object initialization, e.g. in Containers
- If a destructor needs to be defined for a class, typically copy constructor and assignment (“=“) operator need to be specified as well

Copy Constructor

- [OOPs/copy_constructor.cpp at main · ahanapradhan/OOPs]
- A copy constructor initializes an object by copying the member values from an object of the same type
  - If class members are all simple types such as non-pointer values, the compiler-generated copy constructor is sufficient
  - If the class requires more complex initialization, especially pointers, then a custom copy constructor is needed
- Copy constructor is used for copying an object that is passed to a function as an argument or to copy a return value from a called function to the calling function



string s1 = s2; string s1(s2)

Deep vs Shallow Copy

OOPs/deep_shallow_copy.cpp at main · ahanapradhan/OOPs

- If a class member is a pointer then the copy constructor needs to allocate new memory and copy the values from the copy-from object [“deep copy”]
- The compiler-generated copy constructor simply copies the pointer, so that the new pointer still points to the other's memory location [“shallow copy”]

Operator Overloading

- https://github.com/ahanapradhan/OOPs/blob/main/operator_overloading.cpp
- https://github.com/ahanapradhan/OOPs/blob/main/operator_overloading2.cpp

  - Following operators CANNOT be overloaded
    - “.” , “::” and “.*” because they take a name as their second operand
    - “? :” for no particular reason C++ prohibits this
  - Precedence cannot be changed through overloading
    - `a = b * c + d` will always evaluate `b * c` first unless you say `a = b * (c + d)`
  - Overloading is allowed only for user-defined types
    - Operators for built-in types cannot be overloaded

Pre and Post Increment Operators

```
class Counter {
    int value;

public:
    Counter(int v = 0) : value(v) {}

    // Pre-increment: ++obj
    Counter& operator++() {
        ++value;           // increment first
        return *this;      // return updated object
    }

    // Post-increment: obj++
    Counter operator++(int) {
        Counter temp = *this;  // make a copy (old value)
        value++;               // increment afterwards
        return temp;           // return old value
    }

    void display() const {
        cout << "Value: " << value << endl;
    }
};
```

```
int main() {
    Counter c1(5);

    cout << "Initial: ";
    c1.display();

    cout << "After pre-increment (++c1): ";
    (++c1).display();

    cout << "After post-increment (c1++): ";
    (c1++).display();

    cout << "Final state of c1: ";
    c1.display();

    return 0;
}
```

Friend Functions

https://github.com/ahanapradhan/OOPs/blob/main/friend_function.cpp

Friend Class

```
class StopWatch {
        private :
                int start, stop ;
        public:
                StopWatch( ) {
                        start = 0 ;
                        stop = 60;
                }
        friend class Clock ;
};
```

All member functions of class Clock become friend functions

```
class Watch {
        public :
                void show( ) {
                        StopWatch s ;
                        cout << s.start ;
                        cout << s.stop ;
                }
};
```

Can't access private members

```
class Clock {
        public :
                void display( ) {
                        StopWatch s ;
                        cout << s.start ;
                        cout << s.stop ;
                }
};
```

```
int main( ) {
        Watch w ; w.show( ) ;
        Clock c ; c.display( ) ;
}
```

Overloading Operator <<

```
class Comp {
        private :
                double r,i;
        public :
                Comp ( double x, double y ) {
                        r=x;
                        i=y;
                }
                friend ostream& operator << (
                ostream&, comp ); /* cout is an
                                object of ostream class
                                in stream library */
};
```

```
ostream& operator << ( ostream& x, Comp c ) {
        x << c.r <<" " << c.i;
        return x ;
}
```

```
# include <iostream>
using namespace std ;
int main( ) {
        Comp a(1.0,1.0);
        Comp b(2.0,2.0);
        cout <<a << b; /* operator<<(cout, a)
                and operator<<(cout, b) */
}
```

Initializer List

- OOPs/initializer_list.cpp at main · ahanapradhan/OOPs
- c++ - Initializing fields in constructor - initializer list vs constructor body - Stack Overflow

- GDB Cheat Sheet